

Assessing Synthetic Benchmark Reliability

A Case Study on the Canon Layer

Kirill Zemlianskii

June 2026

A wide range of architectural changes:

- Attention: sliding-window, MLA (DeepSeek), gated attention, DSA
- Positional encoding: RoPE, NoPE, ALiBi
- Gating: gated MLP, SwiGLU, MoE

The difficulty in comparing

- Costly training
- Noisy benchmarks
- Overlapping results, with no clear winner overall

Idea: Decompose “reasoning” into atomic skills; design one controlled task per skill.

Advantages:

- Generation on-the-fly, no need for storage
- Controlled difficulty
- Short context
- Cheap enough to run at small scale

The bet: a component that helps on these tasks is worth carrying to scale.

Tasks used in this study

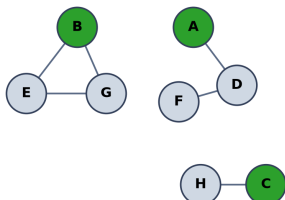
Depo	Reasoning depth
ConComp	Reasoning depth
ShortPath	Reasoning breadth
BFS	Reasoning breadth

Each graph task in two encodings (edges-list / adj-list) $\Rightarrow$ 8 variants.

Task ConComp: Component Factorisation

ConComp: $\rightarrow$ A B C

● component min (answer)



Setup: Undirected random graph (up to 50 nodes, multiple components).
Multi-token node labels.

Format:

`<task> G <sep>  $\rightarrow$  A B C <eos>`

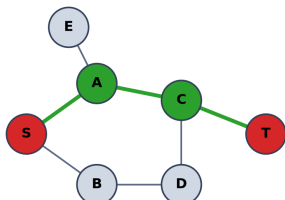
Output r_i : the lexicographically smallest node in component i , one per component in traversal order.

- Partition the graph globally into disjoint components
- Must build a representation of the whole graph

Task ShortPath: Shortest-Path Planning

ShortPath: $S \ T \rightarrow A \ C$

● start / target ● path node (answer)



Setup: Undirected random graph. Start node s and target node t at BFS distance d .

Format:

`<task> G <sep>  $S \ T \rightarrow A \ C$  <eos>`

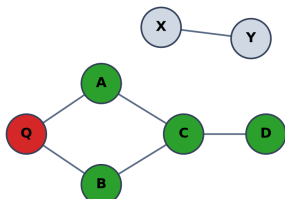
Output the intermediate nodes on the shortest path from s to t (start excluded).

- Step-by-step planning along valid graph edges
- Difficulty set by path length / target distance

Task BFS: Breadth-First Traversal

BFS: (query) $Q \rightarrow A\ B\ C\ D$

● query q ● reachable (answer)



Setup: Undirected random graph. A single query node q .

Format:

<task> G <query> $Q \rightarrow A\ B\ C\ D$ <eos>

Enumerate all nodes reachable from q in BFS discovery order (capped at $n_{\max}$).

- Step-by-step planning along valid graph edges
- Difficulty set by the number of reachable nodes

Sequential tasks (each over a random graph, answer tokens only):

- **ShortestPath**: output a shortest path between two nodes
- **BFS**: output the set of nodes reachable from a query node
- **ConComp**: output one node per connected component

Single answer node tasks (Model produces a single answer node):

- **Depo**: output the k-th successor node of a query node

Two encodings of the same graph:

- **edges_list**: a flat list of edges
- **adj_list**: per-node adjacency

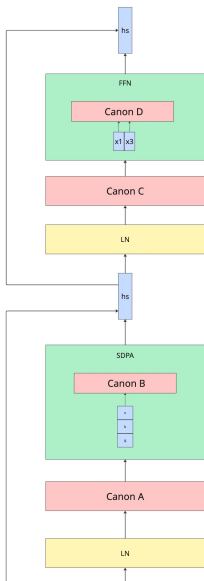
Motivation: Insufficient horizontal mixing in the transformer.

Solution: a lightweight depthwise causal convolution on the residual stream:

$$\tilde{h}_t = \sum_{j=0}^{k-1} w_j \odot h_{t-j} \quad (k \text{ taps, weights shared across channels})$$

Implementation:

- Causal convolution, kernel size k (default 4) + residual standard torch Conv1d init
- Negligible parameter overhead
- Enables horizontal mixing



4 insertion points per transformer block:

- A Pre-attention
- B Post-attention
- C Pre-FFN
- D Post-FFN

The full configuration **Canon-ABCD** places one at every position, and this is the configuration we study throughout.

The original Physics 4.1 work (Allen-Zhu & Li) presents Canon as a broadly useful primitive, validated *on exactly these synthetic benchmarks*, but leaves open *why* it works.

We use Canon as a running example to ask two things:

- Can a single training run on these benchmarks be trusted to rank two architectures?
- Evaluated correctly, is Canon better. What does it change in training dynamics?

⇒ Two questions: measurement reliability first, then mechanism.

Framework: Meta Lingua
(lingua_modified)

Canon: CUDA kernel from
Dao-AILab/causal-conv1d

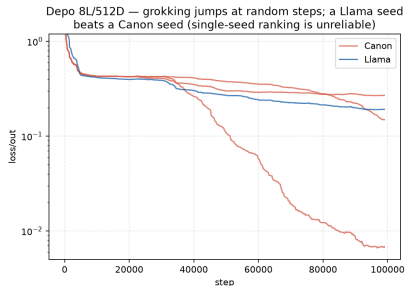
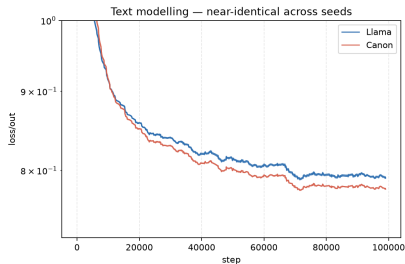
Models: Canon-ABCD vs.
matched Llama (RoPE), identical
width, depth, optimiser, data,
token budget.

Scales: 3M, 10M, 30M, 100M
parameters

Benchmark: 8-task mix
(4 graph tasks $\times$ 2 encodings)

Optimization: AdamW, $3e-4$
LR, 100k steps, cosine
scheduling, $\beta_1=0.9$,
 $\beta_2=0.98$, $\epsilon=1e-6$.

Seed variability

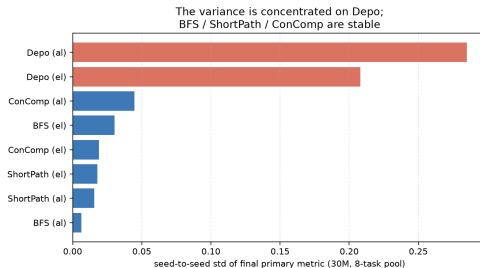


Text: seeds nearly identical

Depo: grokking at random steps

- Text modelling is a stable reference: Canon is reliably better than Llama
- On Depo the grokking occurs at random steps
- Some runs of Canon are worse than baseline Llama

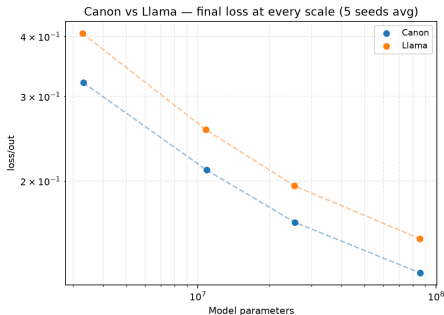
Instability is Linked to Grokking



Final-score spread, by task

- Instability is associated with grokking: tasks that don't grok stay much more stable.

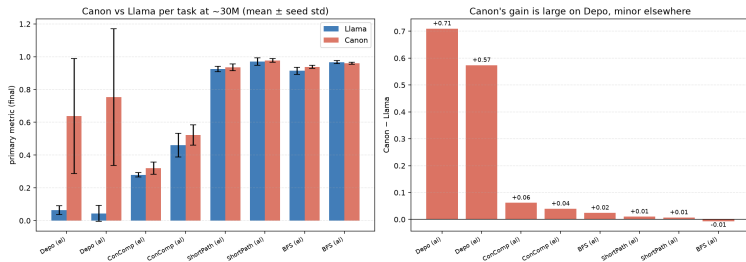
High variance requires more runs; Canon beats Llama



Final loss vs. parameters (5 seeds averaged per point).

- Canon beats Llama at every scale: 3M, 10M, 30M, 100M.
- Required 5 runs per model / size for fair comparison.

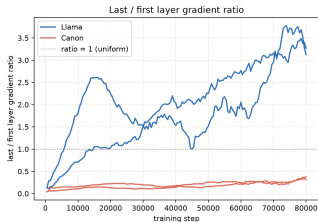
Most significant gain on depth of reasoning tasks



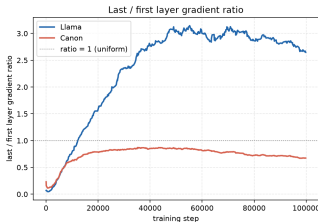
- Depo and ConComp require building representation of the whole graph in one step.
- Those tasks show the largest gain with Canon, while ConComp is much more stable than Depo.
- Depo has the highest correlation with ConComp

Maybe we can use the benchmarks to interpret architectural changes.

Some encouragement:
Some of the qualitative dynamics changes persist between runs and are consistent between text and synthetic.

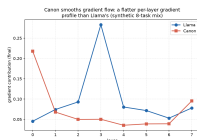


Last/first layer gradient ratio (synthetic, different seeds)

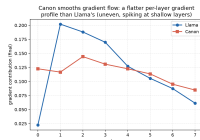


Last/first layer gradient ratio (text)

Gradient profile

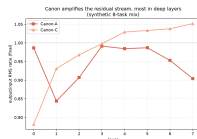


synthetic

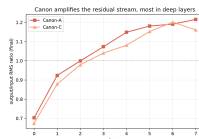


text

Representation (RMS) ratio



synthetic

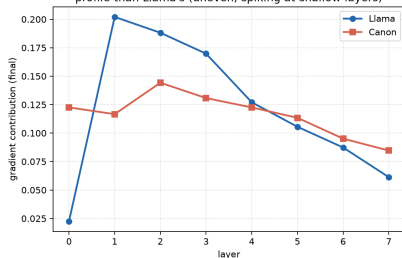


text

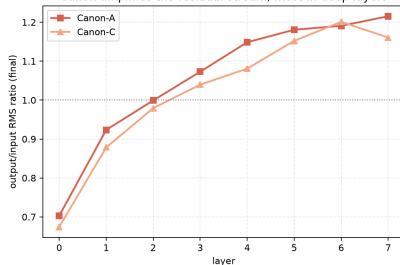
- However, the most interesting mechanistic results we saw on text do not reproduce synthetically
 - Conclusions are hard to draw. Training dynamics on synthetic tasks differ from text.
- ⇒ Further we focus on studying canon on text to gain more reliable insights.

Mechanisms: Gradient Smoothing & Representation Upscaling

Canon smooths gradient flow: a flatter per-layer gradient profile than Llama's (uneven, spiking at shallow layers)



Canon amplifies the residual stream, most in deep layers

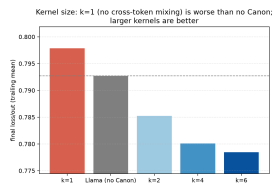
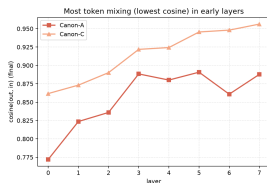
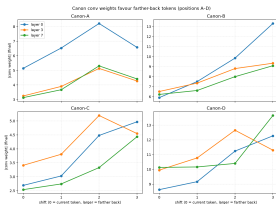


Gradient smoothing: per-layer gradient contribution

- **Gradient smoothing:** Llama spikes at shallow layers and decays; Canon leaves a flatter gradient across layers
- **Representation upscaling:** the Canon conv amplifies the residual stream (RMS ratio > 1), most strongly in the deep layers near the LM head.

Representation upscaling: Canon conv output/input RMS ratio

Mechanism 3: Long-Range Mixing

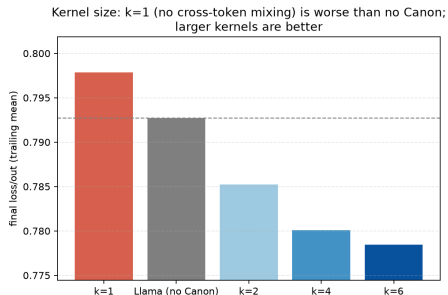


- **Farther-back preference:** Further tokens contribute more tokens (shift 2 > shift 0)
- **Where it acts:** cross-token mixing is strongest in the early layers (lowest output/input cosine).
- **Necessity:** $k=1$ (no cross-token mixing) is worse than baseline; loss improves monotonically as the kernel widens.

- **Gradient redistribution.** Canon rescales the representations and smoothens the gradient across depth.
- **Long-range inductive bias.** Learned weights favour farther-back tokens from initialisation; mixing is strongest in early layers, and removing it ($k=1$) is worse than no Canon at all.

⇒ Additional horizontal information mixing is the primary contribution of Canon.

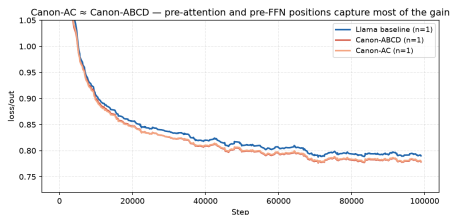
⇒ Depth reasoning tasks benefit most from this.



Sweep $k \in \{1, 2, 4, 6\}$ on text modelling (Llama baseline dashed).

- $k=1$ is worse than no Canon: a pointwise gate that hurts.
- Loss falls monotonically with k , saturating by $k=4$ to 6.
- Cross-token mixing, not extra parameters, drives the gain.

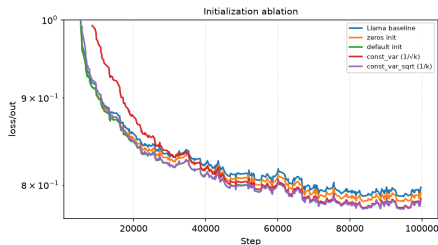
Ablation: Insertion Position (Canon-AC vs. ABCD)



Four insertion points per block (A/B/C/D).

Positions A (pre-attention)
+ C (pre-FFN) recover
essentially the full gain.

Config	Loss
Llama	0.793
Canon-AC	0.782
Canon-ABCD	0.781



All init schemes (zeros, default, const_var, $1/k$) outperform the Llama baseline.

Initializing with mean is the best and better than default.

Optimal configuration from ablations on text:

- **Kernel size** $k \geq 4$. Cross-token mixing is necessary: $k=1$ is worse than no Canon, and the gain saturates by $k \approx 4$ to 6.
- **Positions A + C** (pre-attention + pre-FFN). These recover nearly the full gain of Canon-ABCD;
- **Initialisation: any scheme**. The benefit is init-agnostic

Recommended configuration: **Canon-AC**, $k=4$, **default init**.

The benchmarks

- Synthetic benchmarks are noisy: evaluating architectural changes requires multiple runs, no possibility to reason about the mechanism.
- Variance is tied with grokking
- ConComp is an alternative depth probe to Depo: similar capabilities, correlated with Depo, but far less noisy.

Canon

- Better at every scale (gap largest at small scale); gain concentrated on Depo and ConComp - depth reasoning tasks.
- **Optimal configuration:** Canon-AC, $k=4$, default init. A+C recover the full gain, can be initialized with mean or zero.
- The primary effect of Canon comes from information mixing, not just from scaling.

- The effect of adding horizontal information mixing by increasing the number of layers within the model.
- Design of robust synthetic tasks for depth reasoning and their alignment with text modeling.

Thank you!